Manuel Développeur : Simulateur d'Environnement Drone

Sommaire

1. Introduction....1
2. Architecture Globale....1
3. Définition du MDP (Processus de Décision Markovien)....2
    3.1. L'Espace d'États....2
    3.2. L'Espace d'Actions....2
    3.3. Fonction de Transition....3
4. Intégration avec un Algorithme d'Apprentissage....3
    4.1. Instanciation programmatique....3
5. Précautions et Limitations....3

1. Introduction

Ce document décrit l'architecture et l'utilisation de l'environnement de simulation de vol de drone. Cet environnement simule un drone évoluant sur une grille avec une gestion de batterie, des zones de danger, et des mécanismes de décollage, d'atterrissage et de recharge.

Le code est écrit en langage python et ne requiert pas de modules en particulier pour fonctionner. Il est structuré autour d'une approche orientée objet qui modélise l'environnement, l'état interne du drone et les règles de transition. Le code a été conçu de manière conceptuelle pour modéliser des Processus de Décision Markoviens (MDP). Il expose de manière native les états, les transitions et les contraintes physiques du drone, ce qui le rend idéal pour l'entraînement d'algorithmes d'Apprentissage par Renforcement ou de planification.

2. Architecture Globale

Le code est découpé en plusieurs classes :

Couple : Représente des coordonnées 2D (x,y). Implémente __eq__ et __hash__ pour être utilisable comme clé dans des dictionnaires ou des ensembles.

Etat : Encapsule les variables d'état du système à un instant t (pos, battery, airborne, docked, crashed). C'est la structure de données fondamentale pour les algorithmes d'apprentissage.

Zone : Représente la grille géographique, gère les dimensions (maxX, maxY) et la division du terrain.

Drone : Maintient l'état interne de l'agent physique (batterie, position, statut de vol).

Environnement : C'est le cœur du simulateur, il contient la logique métier, les actions possibles et génère les transitions d'état.

Graph : Fournit des outils d'exploration d'espace d'états (ex: parcours en profondeur itératif) pour générer l'ensemble des états atteignables.

3. Définition du MDP (Processus de Décision Markovien)

Pour coupler ce code avec un algorithme d'apprentissage, il faut modéliser le problème selon l'Espace d'États, l'Espace d'Actions, et la Fonction de Transition.

3.1. L'Espace d'États

L'état de l'environnement peut être récupéré à tout moment via env.drone.get_etat(). Il retourne une instance de la classe Etat contenant :

pos (Couple) : Les coordonnées (x, y) du drone.

battery (int) : Niveau de batterie de 0 à 100.

airborne (int) : Booléen (1 = en vol, 0 = au sol).

docked (int) : Booléen (1 = amarré à la base, 0 = non amarré).

crashed (int) : Booléen (1 = détruit, 0 = opérationnel).

Note pour le ML : La classe Etat implémente __hash__ et __eq__. Elle peut donc directement utiliser ces objets comme clés dans un dictionnaire pour le RL tabulaire.

3.2. L'Espace d'Actions

Un agent intelligent peut déclencher les actions suivantes en appelant les méthodes de la classe Environnement:

1. **takeoff_cmd()** : Fait décoller le drone (nécessite battery > 0).
2. **move_step(politique)** : Déplace le drone. Consomme 1 point de batterie.
3. **avoid_maneuver(politique)** : Mouvement sécurisé évitant les zones de danger. Consomme 2 points de batterie.

4. **return_home()** : Mouvement d'une case vers la base. Consomme 1 point de batterie.
5. **dock_cmd()** : Amarre le drone s'il se trouve sur sa base et en vol.
6. **undock_cmd()** : Désamarre le drone et le met en vol.
7. **charge_step()** : Recharge 5 points de batterie (nécessite d'être amarré).
8. **emergency_stop()** : Force le crash du drone (état terminal).

3.3. Fonction de Transition

Pour le Model-Based RL ou la planification, l'environnement fournit une fonction générative : env.successeurs(etat: Etat) -> list[Etat] Cette méthode calcule de manière déterministe tous les états suivants légaux et possibles depuis un état donné, sans altérer l'environnement en cours.

4. Intégration avec un Algorithme d'Apprentissage

Le script actuel contient une boucle while interactive basée sur des input(). Pour automatiser l'apprentissage, il faut contourner ou supprimer cette boucle située en bas du fichier et piloter l'environnement via une boucle propre.

4.1. Instanciation programmatique

Au lieu d'utiliser les inputs consoles de l'utilisateur, il faut initialisez l'environnement manuellement dans le script RL.

5. Précautions et Limitations

1. **Obstacles fatals** : Le code précise if next_pos in self.danger: self.drone.obstacle_distance = 0. Dans move_step, atterrir sur un obstacle ne déclenche pas le crash (*crashed=1*) directement par défaut à moins de rater la liste des mouvements valides (via sequence). Il faut décider si l'algorithme d'apprentissage considère qu'un obstacle_distance = 0 équivaut à un état terminal done = True.
2. **Fonction de Récompense** : L'environnement simule la physique et les contraintes, mais n'émet aucun signal de récompense. La conception de la fonction de récompense incombera entièrement à la couche qui englobera ce code.